NOVA SCHOOL OF SCIENCE & TECHNOLOGY

STR

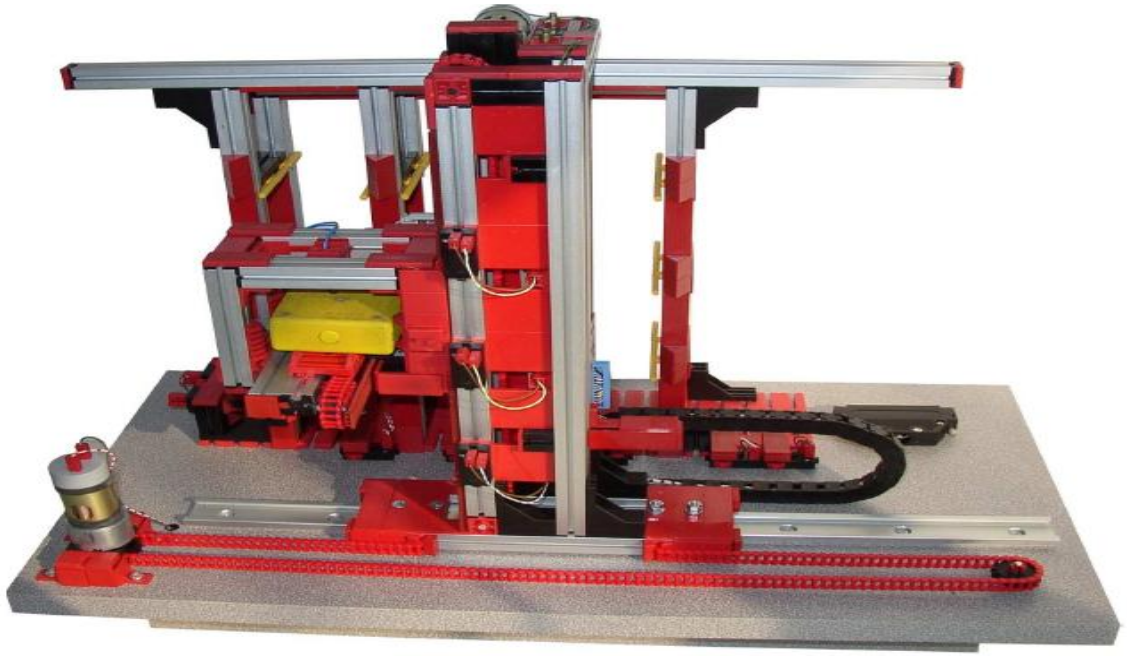
LAB WORK 2: CLASS 2

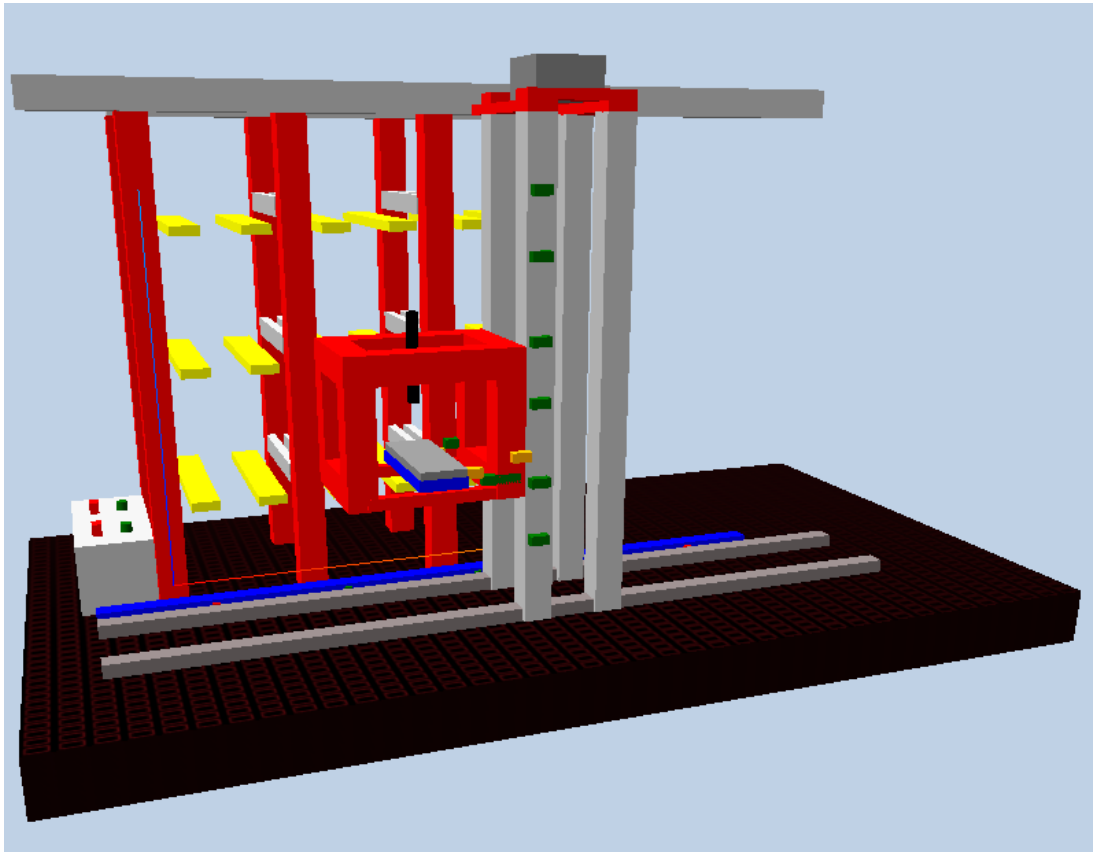
JNI Development for Interacting with the Physical System

JAVA Real-Time

2025- 2026

LABWORK 2 – COMPACT STORAGE KIT





OBJECTIVES

PART 1:
JNI Development

- Creation of the JNI interface
- Accessing low-level functionality
- Testing from Java Application

PART 2:
Java Real-Time

- The Axis Interface
- Developing class Mechanism
- Identification of needed Java Threads

PART 1

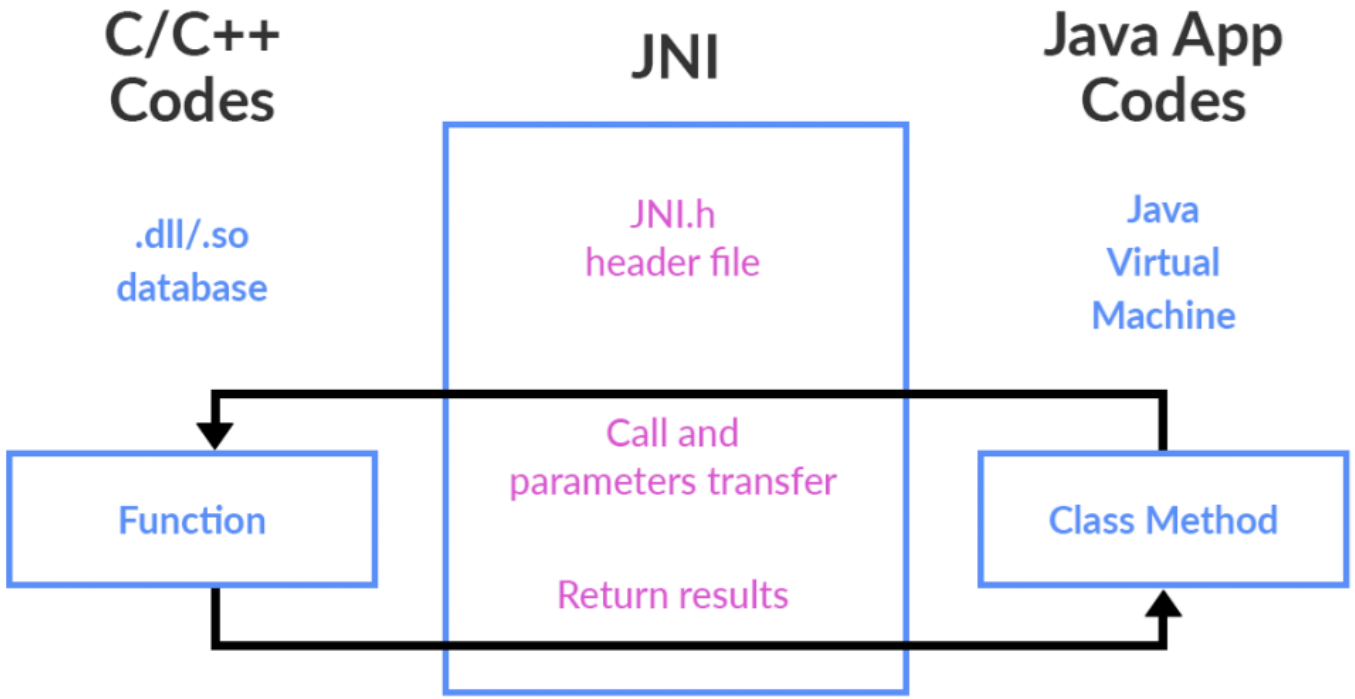
JNI development for interacting with the physical system

CREATION OF THE JNI INTERFACE

We are now ready to proceed to the next phase, JNI interoperability, for utilization within the Java Application

While you can write applications entirely in Java, there are situations where Java alone does not meet the needs of your application. Programmers use the **JNI** to write Java native methods to handle those situations when an application cannot be written entirely in Java.

https://docs.oracle.com/javase/7/docs/technotes/guides/jni/spec/intro.html

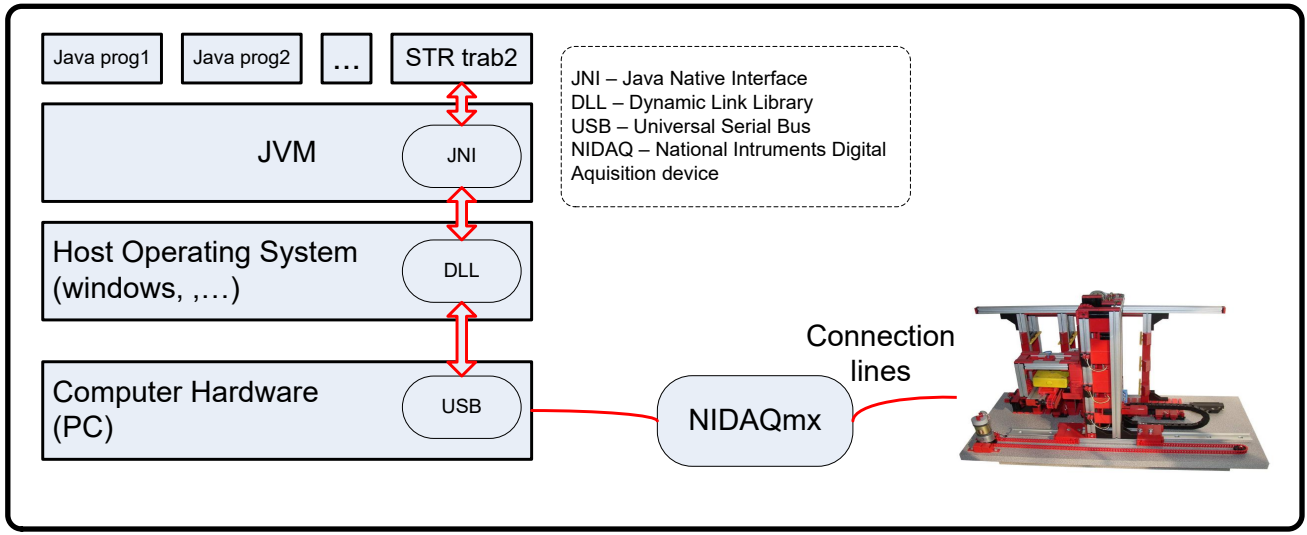


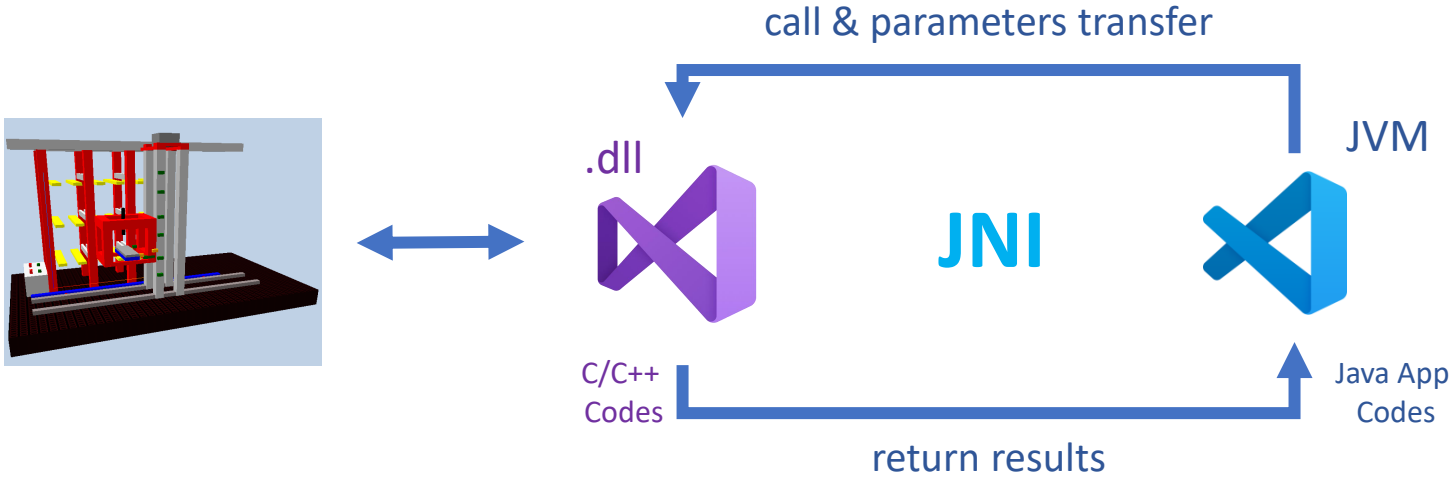
https://redwerk.com/blog/3-reasons-why-we-love-jni/

CREATION OF THE JNI INTERFACE

Java Application

- Create a **Java Application** using the **VS Code** environment
- Proceed with the development of the JNI mechanism
- Create a demo **class** to verify that the Java Program can interact with the Compact Storage





CREATION OF THE JNI INTERFACE

Signatures in Java naming convention (complete table):

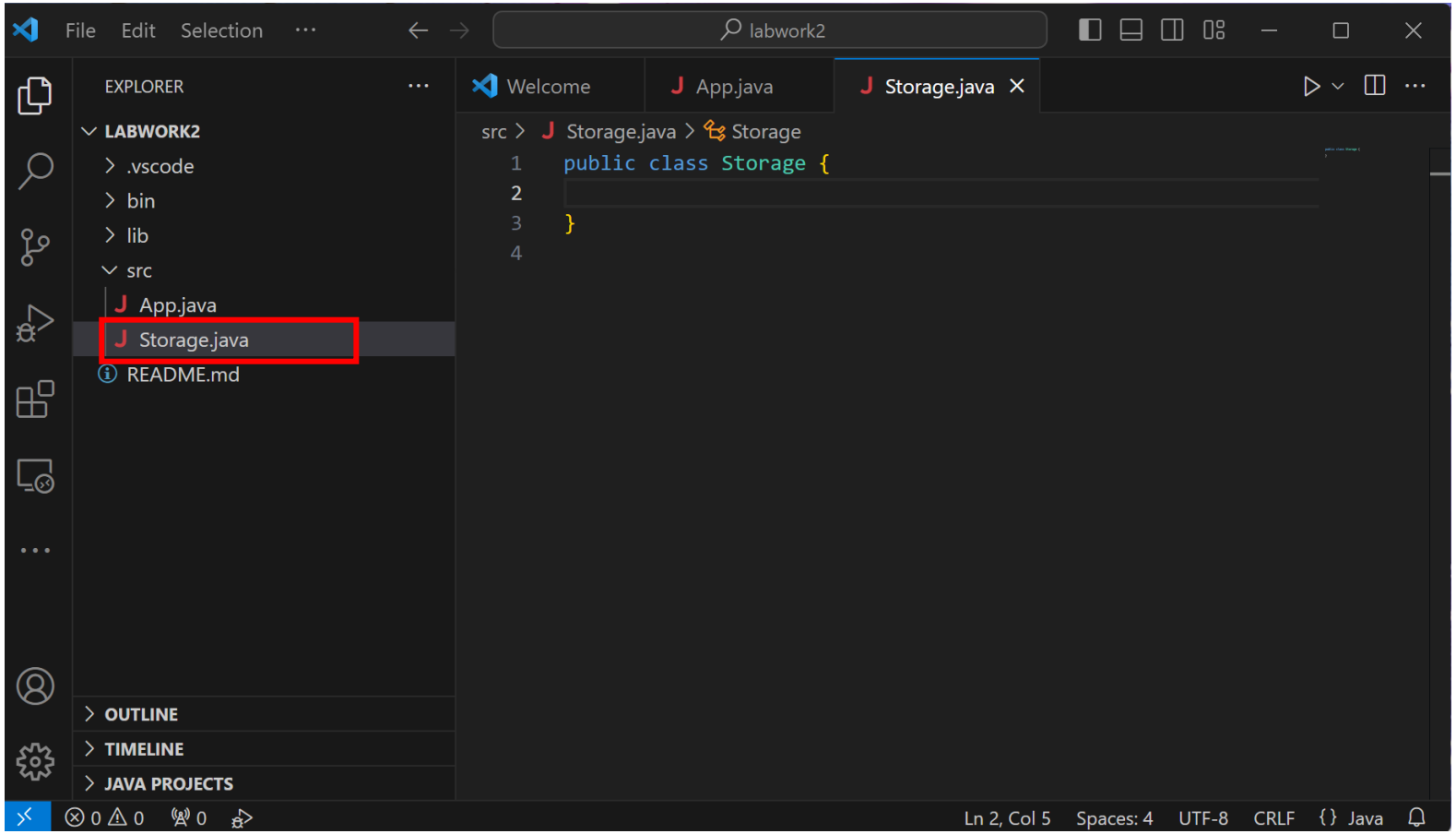
Sensors signature functions in C	In Java
int getXPos();	native public int getXPos();
int getZPos();	native public int getZPos();
int getYPos();	native public int getYPos();
(complete the table)	

Actuator signature functions in C	In Java
void initializeHardwarePorts();	native public void initializeHardwarePorts();
void moveXRight();	native public void moveXRight();
void moveXLeft();	native public void void moveXLeft();
void stopX();	native public void stopX ();
(complete the table)	

Copy the Java signatures from the tables into a new class **Storage** (see next slide).

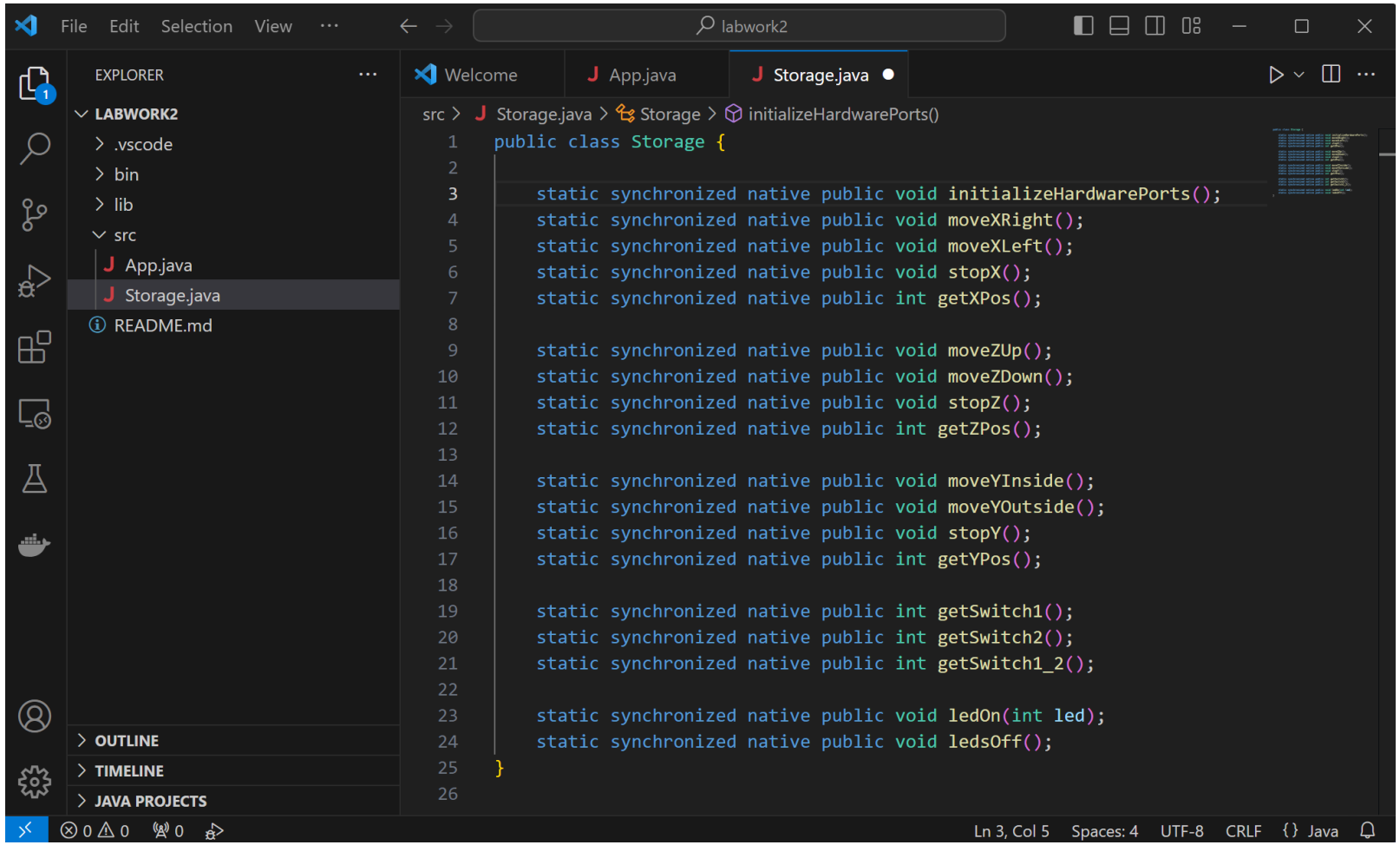
CREATION OF THE JNI INTERFACE

- Open the VS Code **labwork2** project from last week.
- Create a new java class named **Storage.java**:
  - Right-click in \str folder, and select New File



CREATION OF THE JNI INTERFACE

- ❑ Copy the signatures from previous table to Storage class.
- ❑ Ensure file is saved.



CREATION OF THE JNI INTERFACE

Generate Heading File

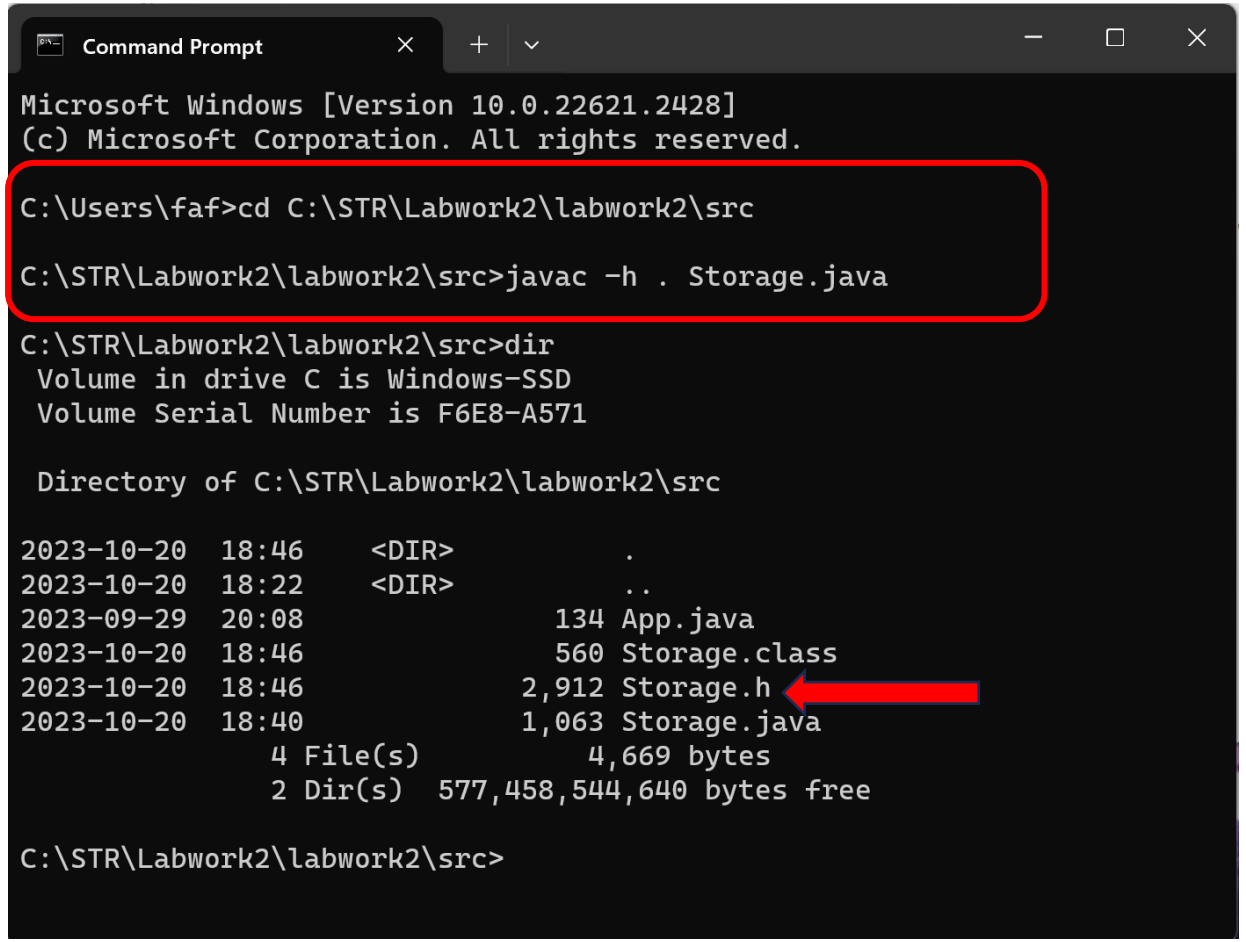
- Launch a console cmd.exe
- Considering the working directory (C:\STR\Labwork2), execute the command:

  C:\STR\Labwork2\labwork2\src

- Then execute:

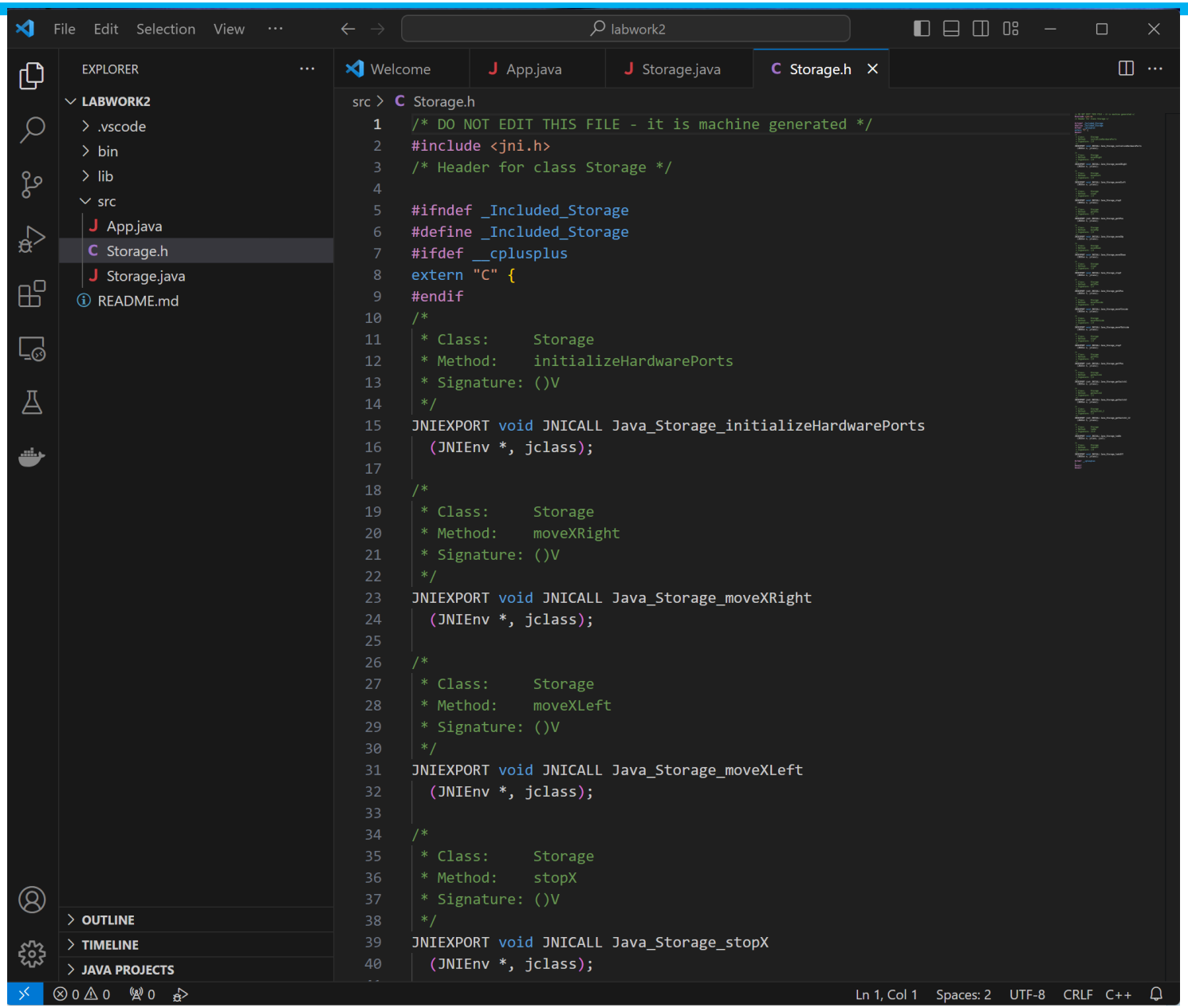
  javac -h . Storage.java
  dir
  del *.class

- As you can see, the file **Storage.h** has been created, which needs to be included into the Visual Studio Project.



CREATION OF THE JNI INTERFACE

The obtained signatures



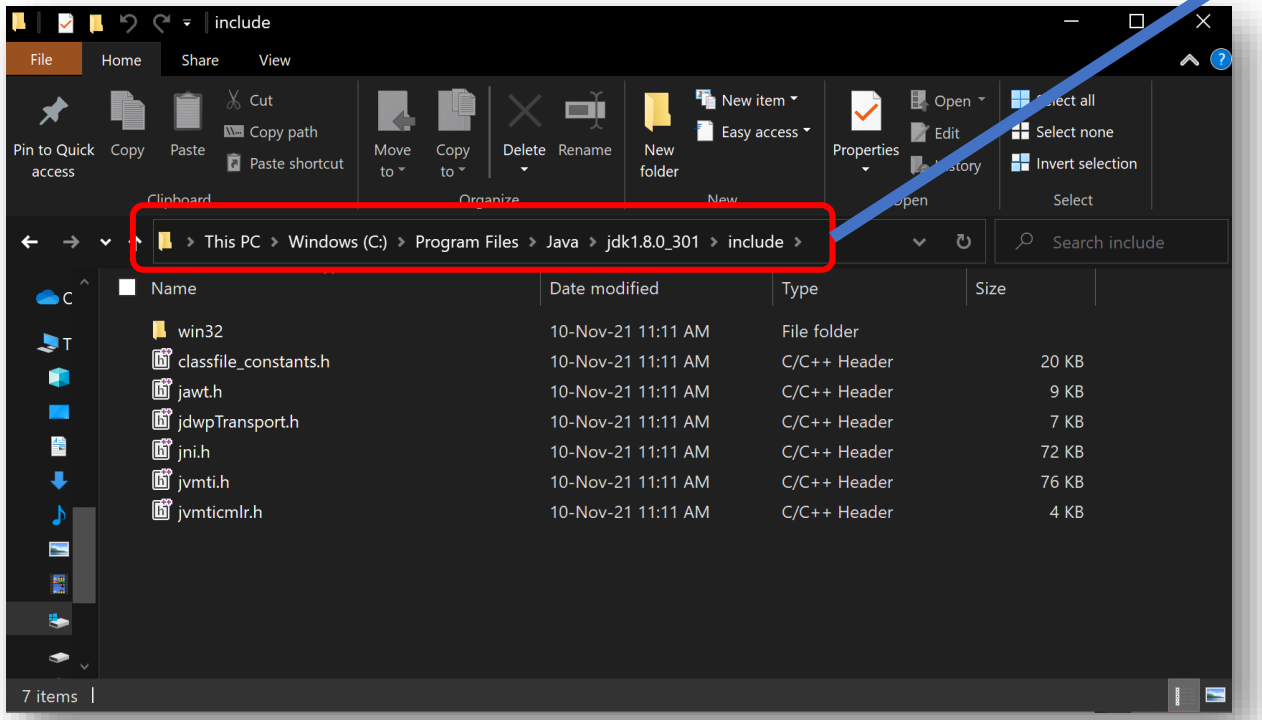
CREATION OF THE JNI INTERFACE

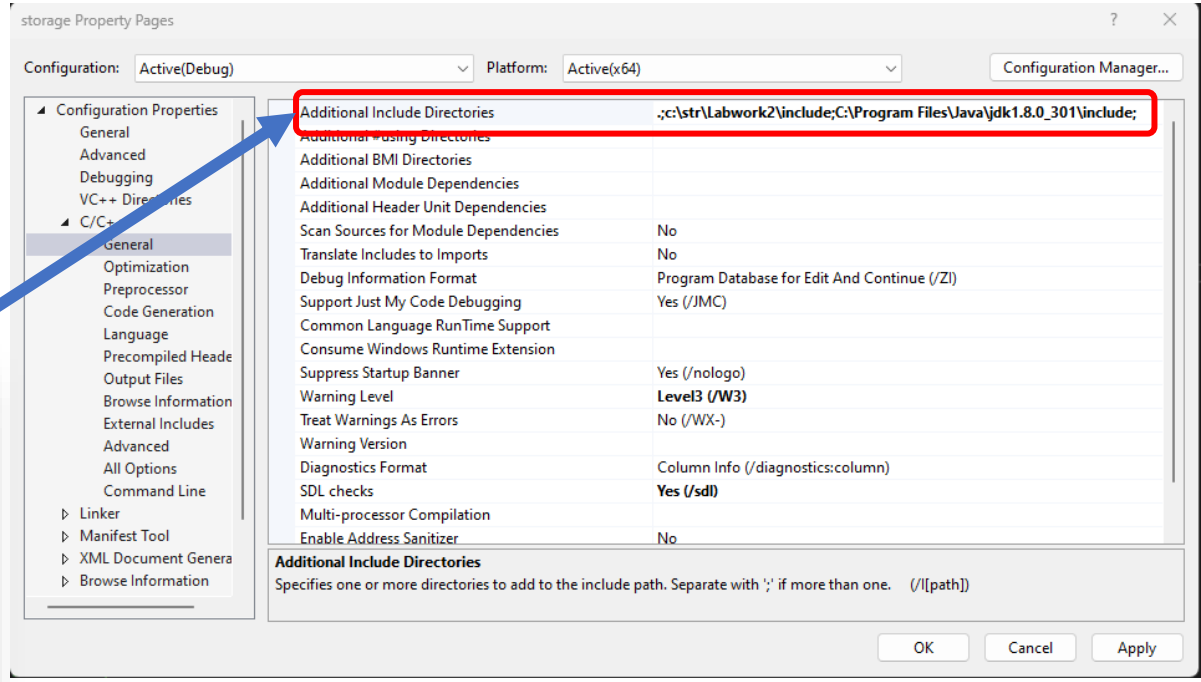
In Visual Studio Project **storage** add the Java include path into the Visual Studio project properties.

Must be something like this e.g. (depends on java version and installed path:

C:\Program Files\Java\jdk-XXX\include;
C:\Program Files\Java\ jdk-XXX\include \win32

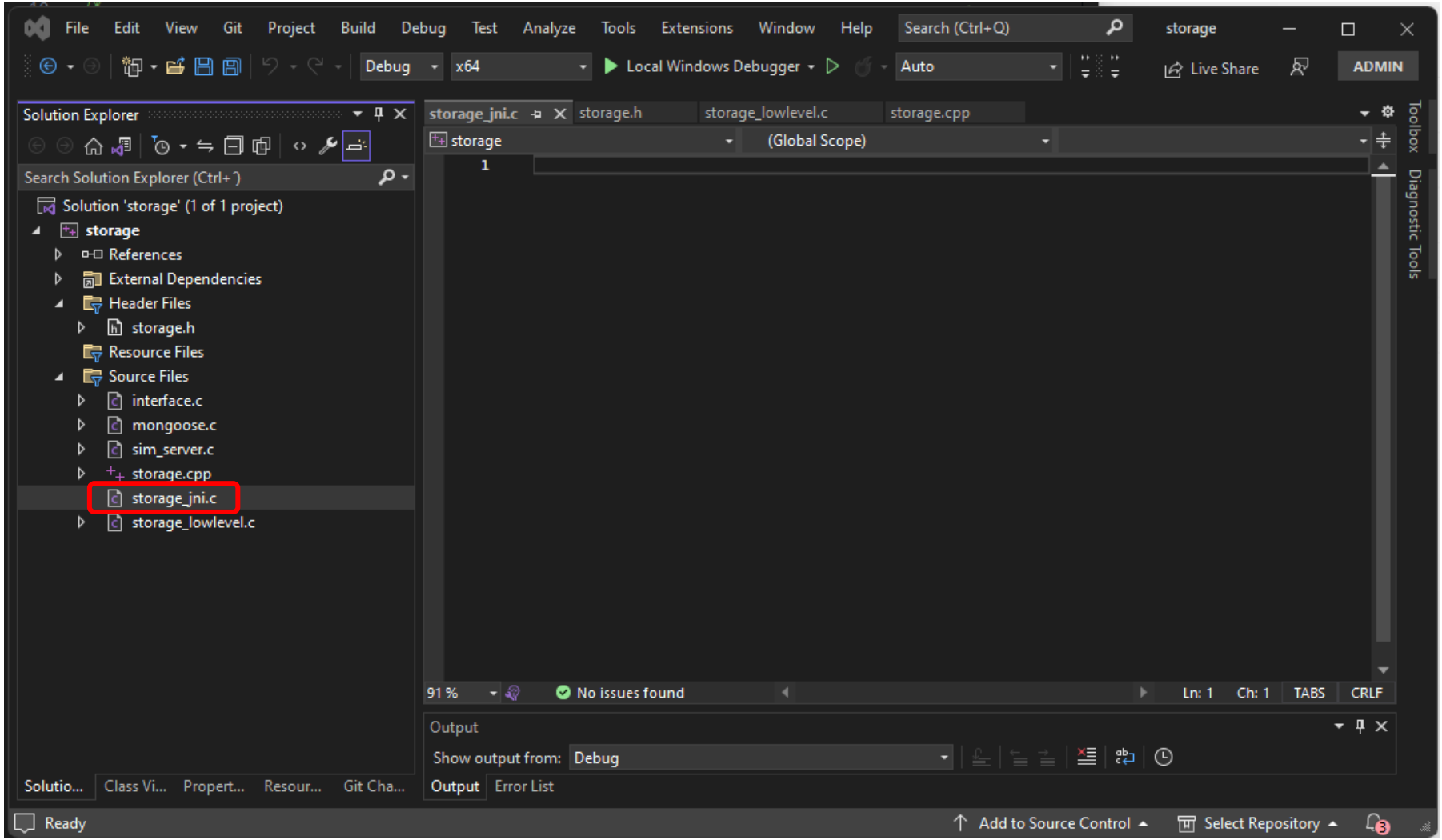
Split each path with a semi-colon (;)





CREATION OF THE JNI INTERFACE

Add new file **storage_jni.c** into your VS project

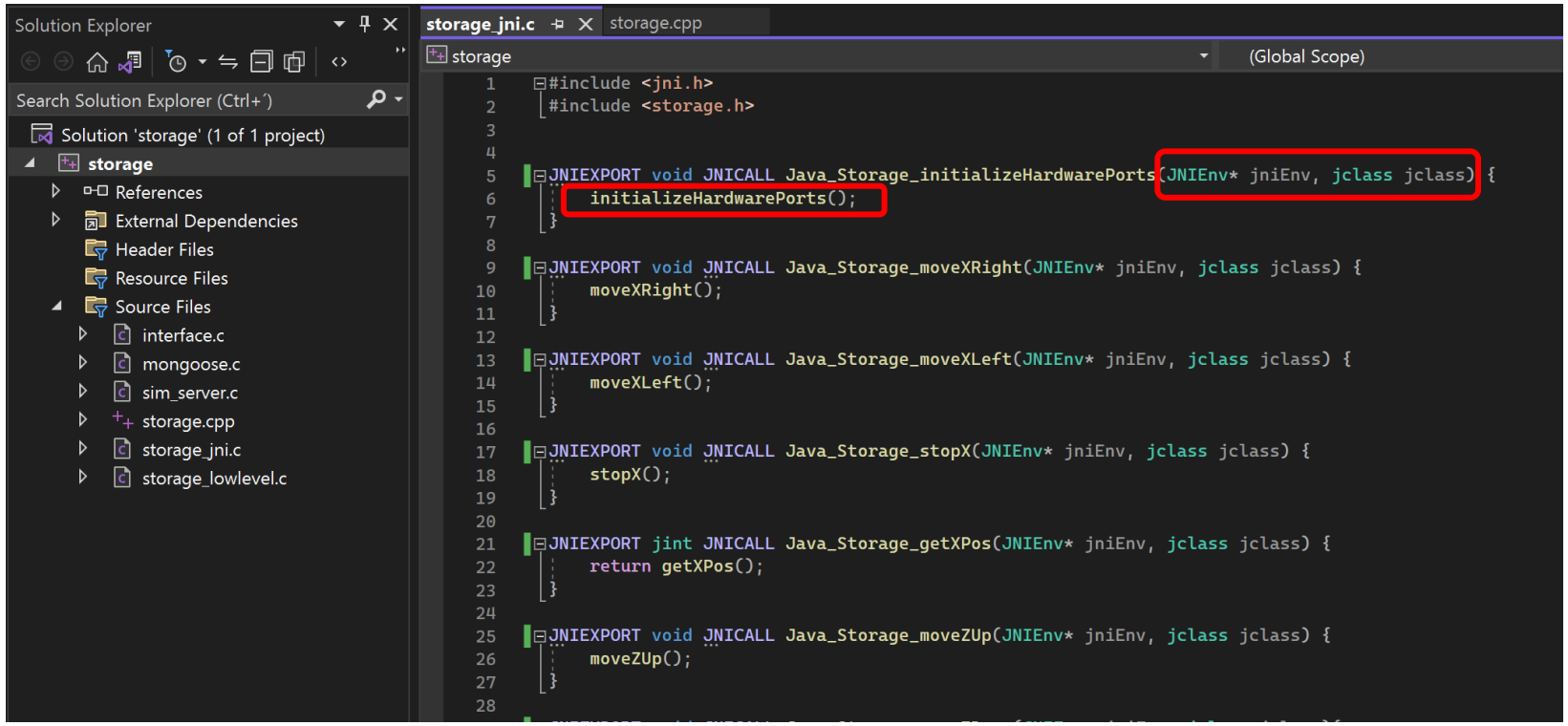


CREATION OF THE JNI INTERFACE

Copy the generated low-level functions from **Storage.h**

Complete as illustrated in the picture for all the functions

Functions called from the JNI stubs are specified in **storage.h** and developed in **storage_lowlevel.c**

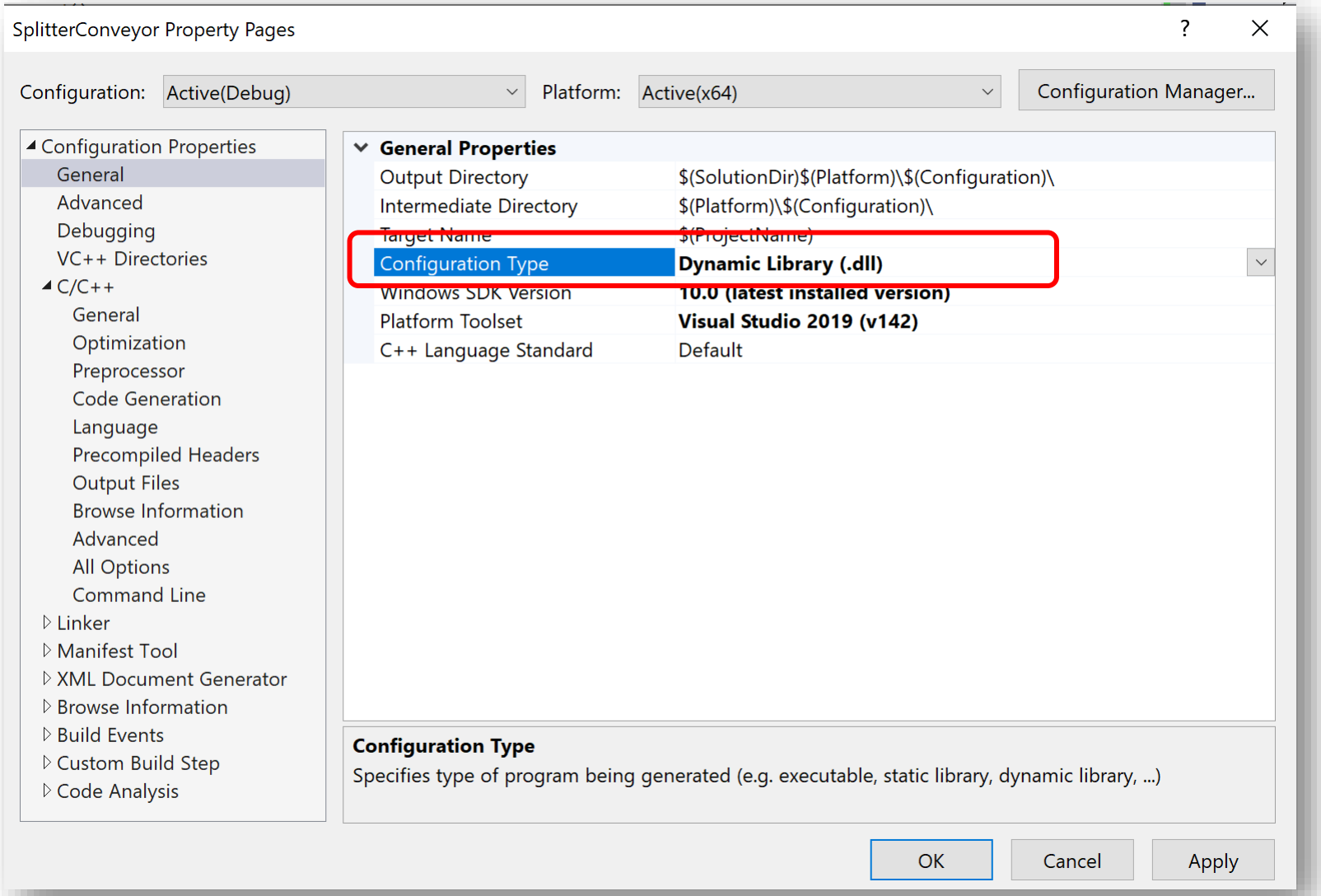


Attention to function ledOn!!!

```
JNIEXPORT void JNICALL Java_Storage_ledOn(JNIEnv* jniEnv, jclass jclass, jint n) {
    ledOn(n);
}
```

CREATION OF THE JNI INTERFACE

Transform Visual Studio project into **DLL**

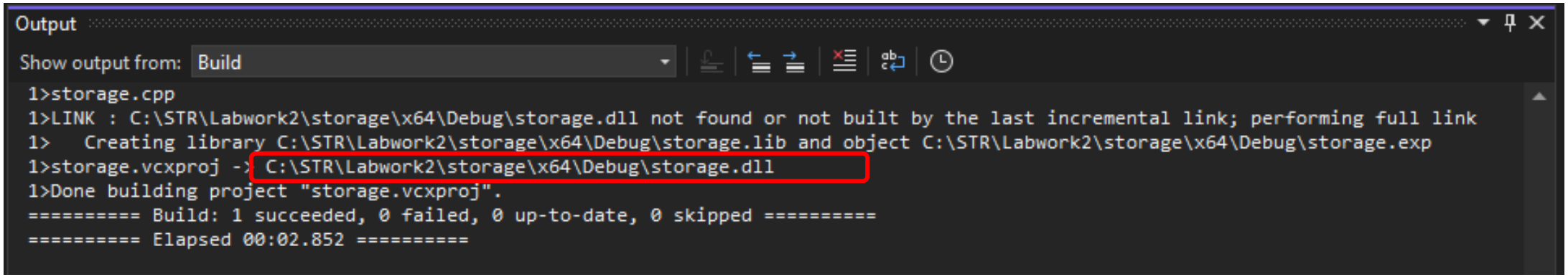


Creation of the JNI interface

Build the project

Build the project and…

Hint: **don’t try run it** (not an executable *.exe anymore)



After that, copy the path for the generated DLL, as shown in the figure.

CREATION OF THE JNI INTERFACE

Code to load DLL from Java

Copy-past the DLL path inside **Storage.java** class

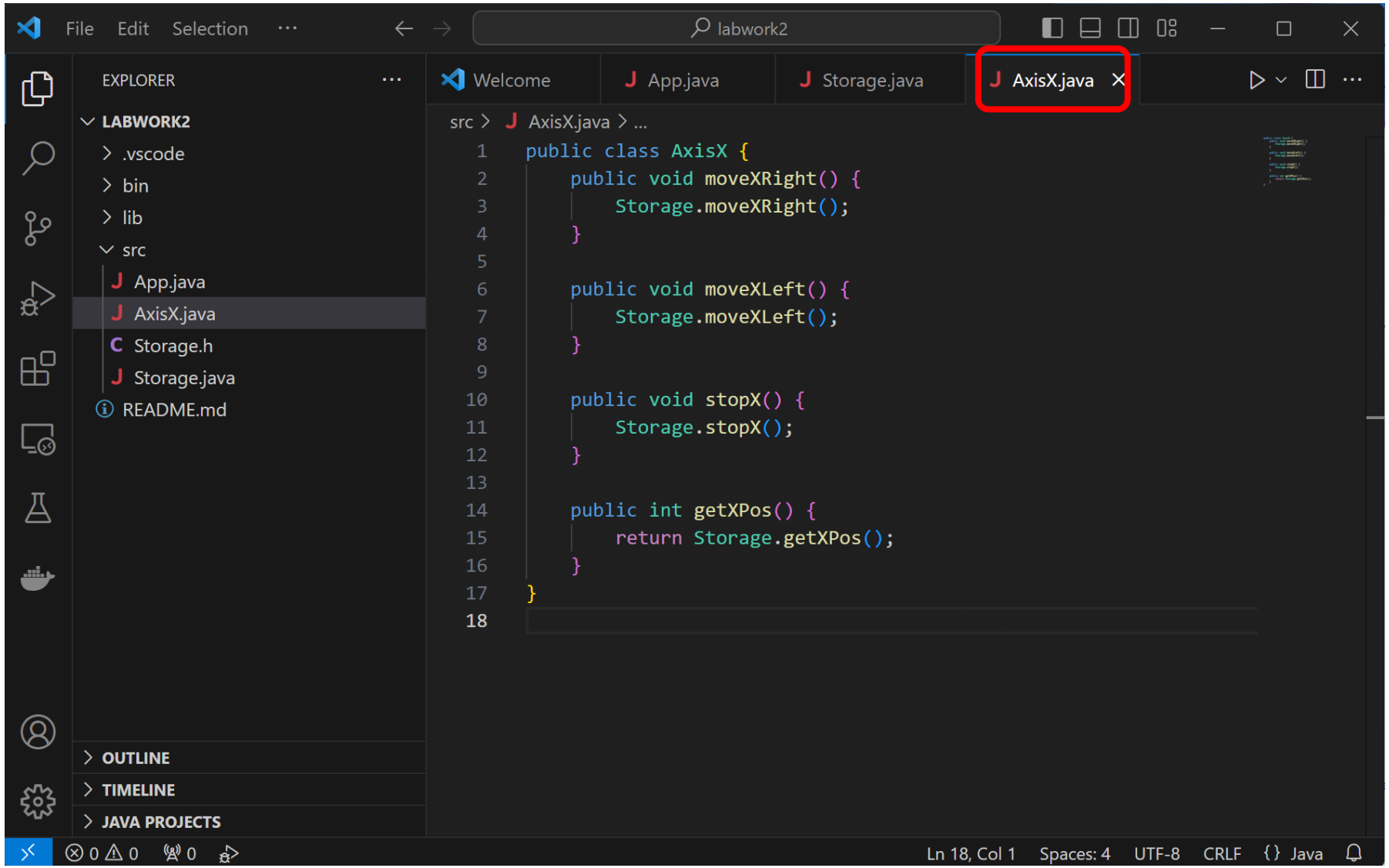
```java
public class Storage {

    static{
        System.load(filename:"C:\\STR\\Labwork2\\storage\\x64\\Debug\\storage.dll
    }

    static synchronized native public void initializeHardwarePorts();
    static synchronized native public void moveXRight();
    static synchronized native public void moveXLeft();
    static synchronized native public void stopX();
    static synchronized native public int getXPos();

    static synchronized native public void moveZUp();
    static synchronized native public void moveZDown();
    static synchronized native public void stopZ();
    static synchronized native public int getZPos();

    static synchronized native public void moveYInside();
    static synchronized native public void moveYOutside();
    static synchronized native public void stopY();
    static synchronized native public int getYPos();

    static synchronized native public int getSwitch1();
    static synchronized native public int getSwitch2();
    static synchronized native public int getSwitch1_2();

    static synchronized native public void ledOn(int led);
    static synchronized native public void ledsOff();
}
```

Hint: Write the empty string (“”) first, then copy-past. This way, double backslashes are added automatically.

ACCESSING LOW-LEVEL FUNCTIONALITY

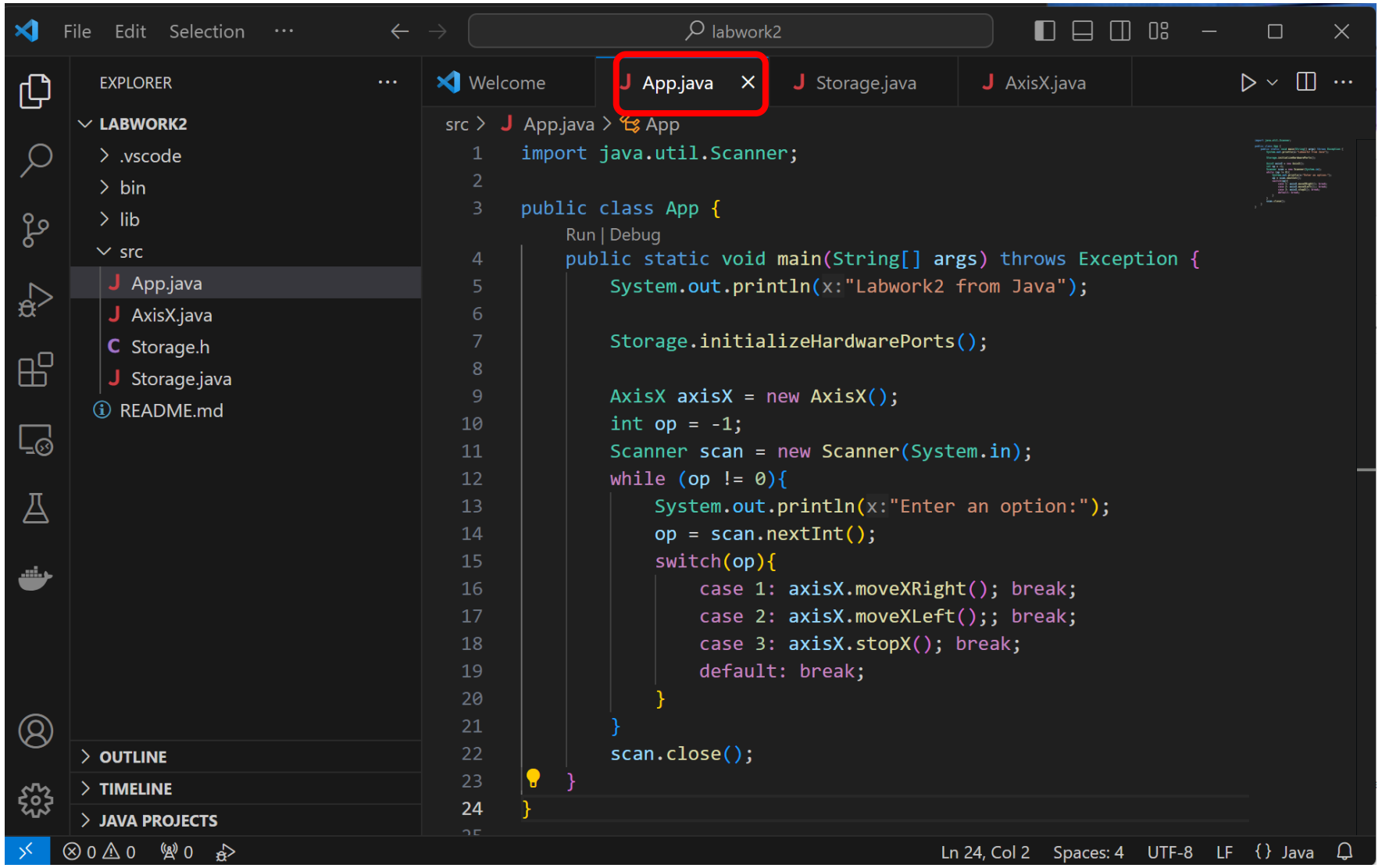
Add new Java class (1): Add a new class named **AxisX.java**

Copy the methods illustrated in the figure



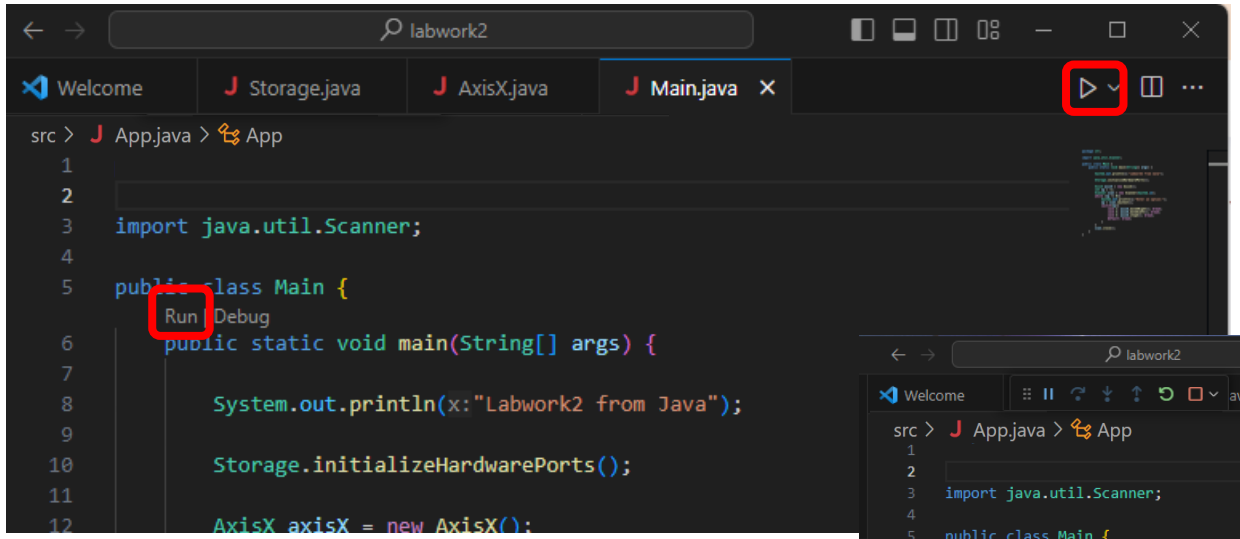
Testing from Java Application

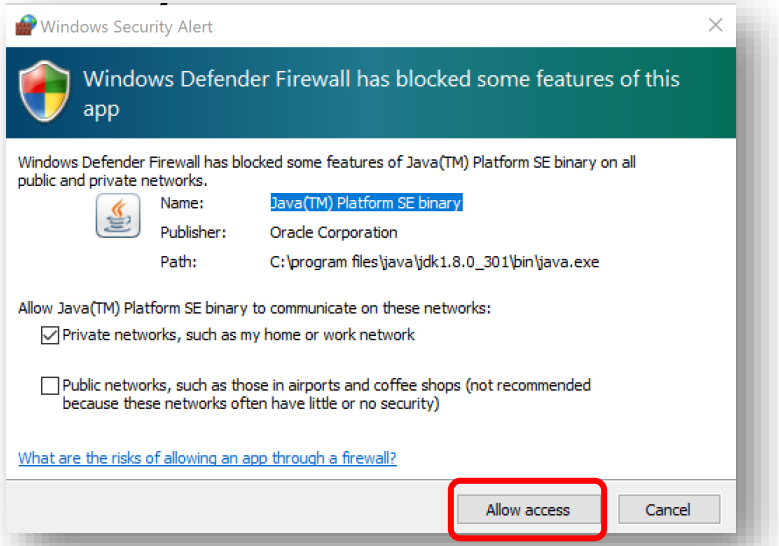
Open App.java class

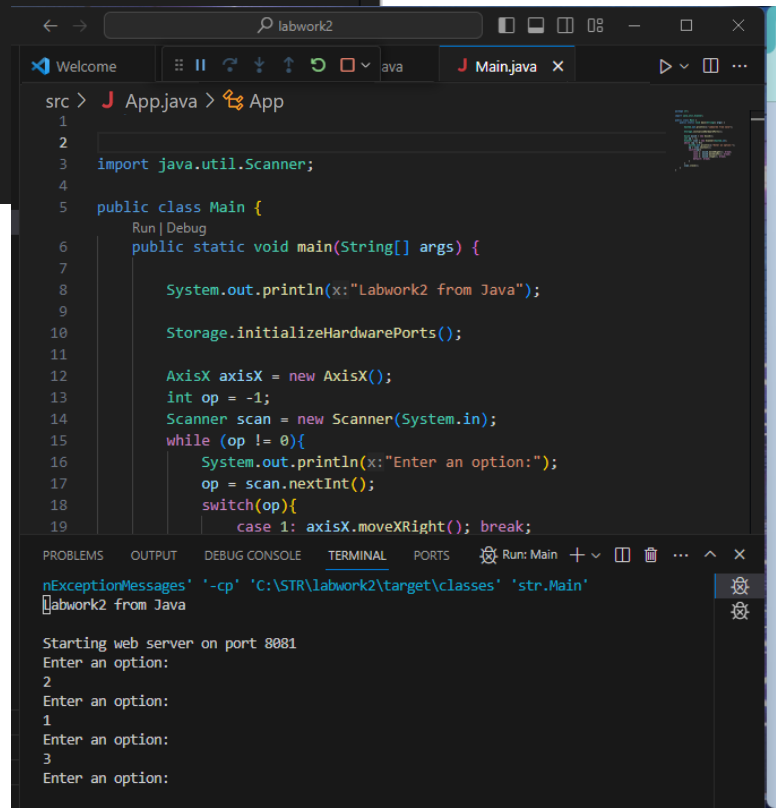


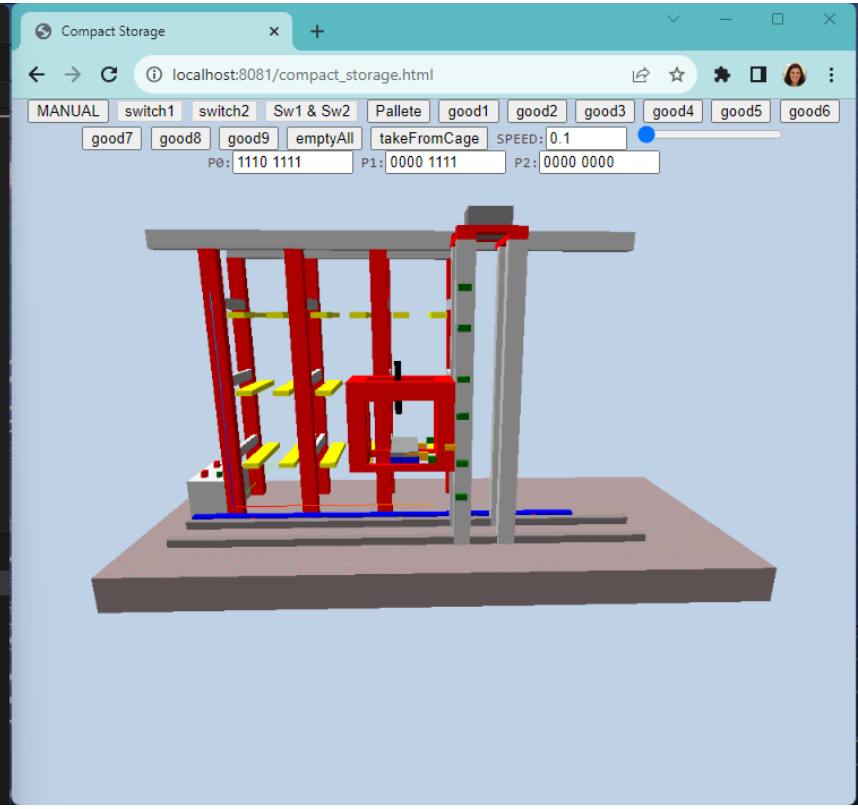
TESTING FROM JAVA APPLICATION

Testing: Run App.java (2 ways of running as depicted below)









TESTING FROM JAVA APPLICATION

If you get unsatisfied link error

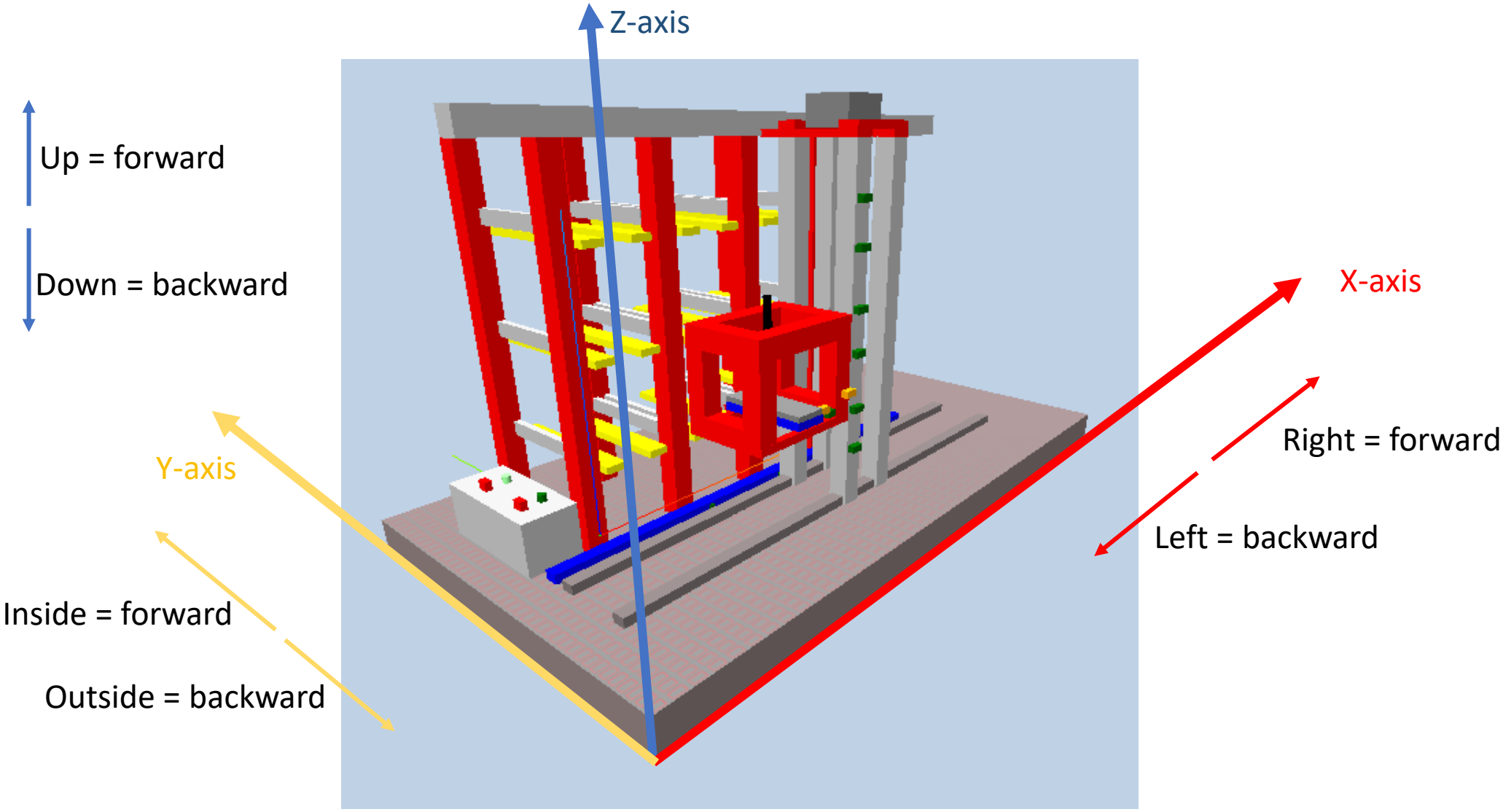
Well, that's bad luck…
We need to repeat the process of using the Storage through the JNI interface.
Repeat the steps of this process very carefully.

```
Output - labwork2 (run)
Exception in thread "main" java.lang.UnsatisfiedLinkError:
        at
        at
        at
C:\Users\aio\AppData\Local\NetBeans\Cache\12.5\executor-snippets\run.xml:111: The following error occurred while executing this line:
C:\Users\aio\AppData\Local\NetBeans\Cache\12.5\executor-snippets\run.xml:94: Java returned: 1
BUILD FAILED (total time: 11 seconds)
```

PART 2

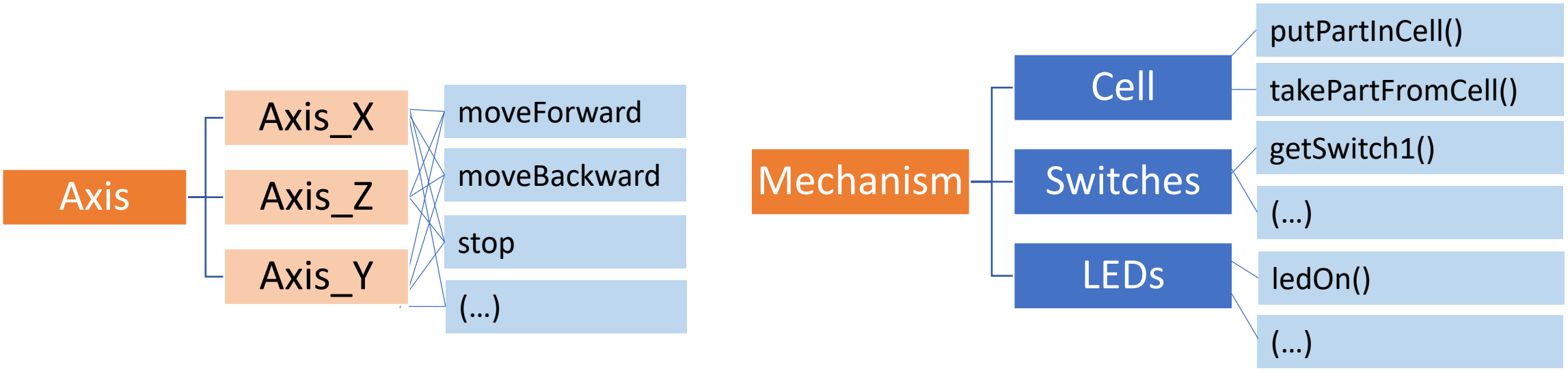
Java Real-Time

LABWORK 2 – AXIS ABSTRACTION



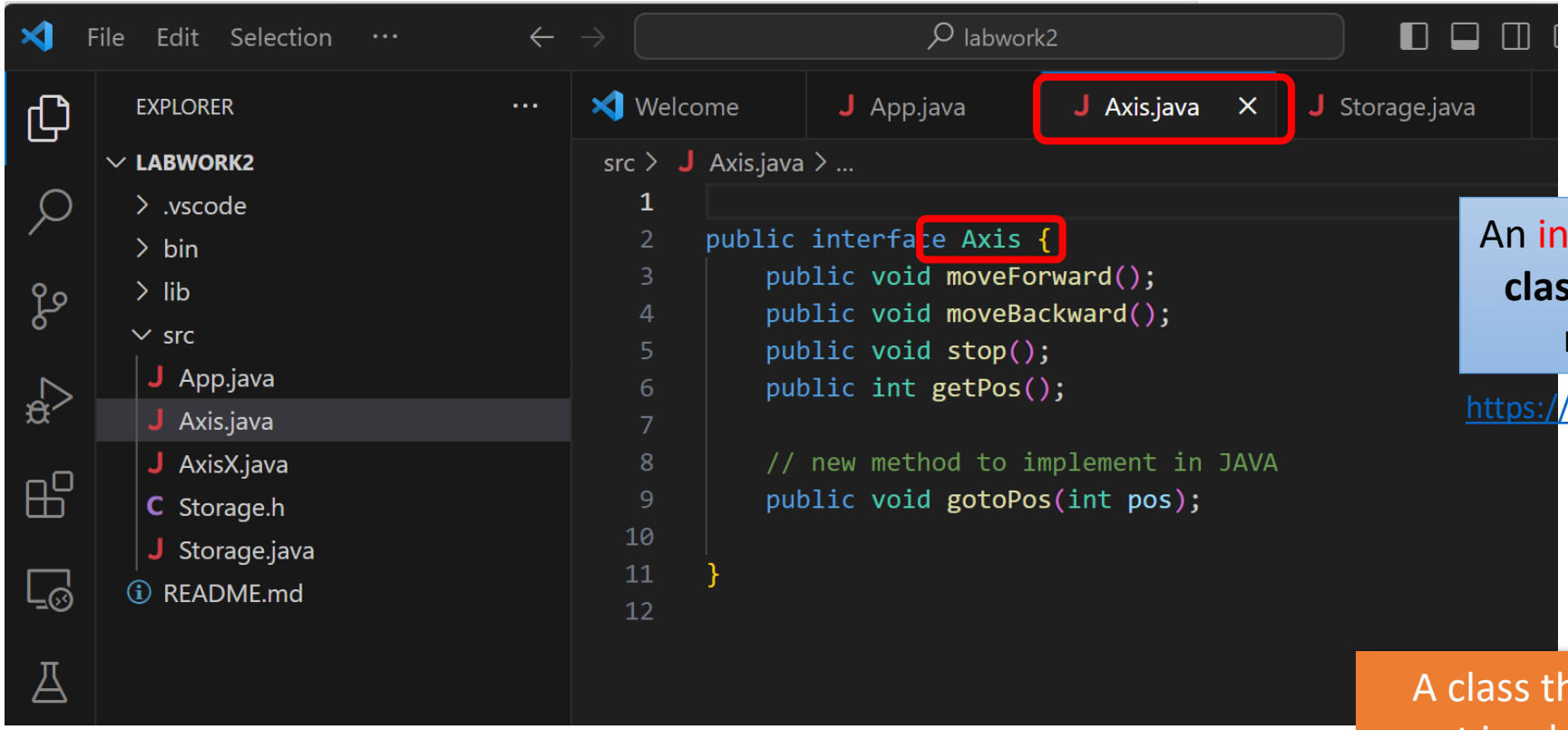
THE JAVA SIDE OF JNI

- Having the class **Storage.java** already developed (the DLL), at the Java side we need to organize our classes into a more logical and straightforward way.
- This should be done by aggregating the storage functionality as several well identified classes, some of them as Java threads.
- For instance, we can define concrete classes for axis operation and a class named mechanism, which aggregates the functionality for the cell movements, the switches, and the LEDs.



THE AXIS INTERFACE

Let's start by providing an interface definition for the Axis: Java interface -> Axis.java



An interface is a completely **"abstract class"** that is used to group related methods with empty bodies

https://www.w3schools.com/java/java_interface.asp

A class that implements a specific interface must implement the methods of the interface.

The interface has several methods that are used in each axis.
Although each axis has distinct implementation, in an abstract way they behave similarly…..
As such, interface **Axis** specifies and describes the behavior of each axis in an abstract way.

THE “NEW”AXISX CLASS

Welcome | App.java | Axis.java | Storage.java | AxisX.java

src > AxisX.java > ...

```java
public class AxisX implements Axis{
    @Override
    public void moveForward() {
        Storage.moveXRight();
    }

    @Override
    public void moveBackward() {
        Storage.moveXLeft();
    }

    @Override
    public void stop() {
        Storage.stopX();
    }

    @Override
    public int getPos() {
        return Storage.getXPos();
    }

    @Override
    public void gotoPos(int pos) {
        // TODO Auto-generated method stub
        throw new UnsupportedOperationException(message:"Unimplemented method 'gotoPos'");

        //to be developed in JAVA
    }
}
```

Each concrete **axis** can follow the **Axis interface** and implement the concrete functionality of each Axis’s methods.

When a class implements an axis, it must **implement all** the interface’s methods.

Based on the given interface, try develop **three classes** for each **axis** existing in the physical system.

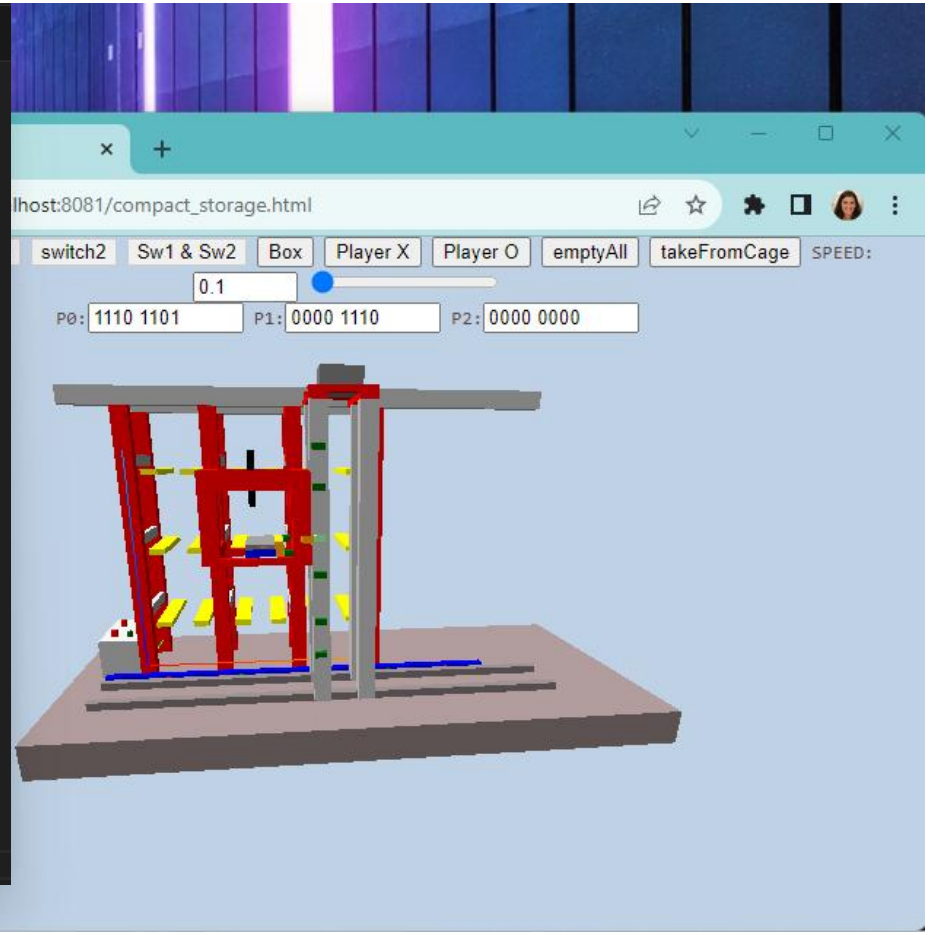
Testing from Java Application

App.java class -> Change it accordingly with the new Axis methods and test

Welcome | App.java | Axis.java | Storage.java | AxisX.java

src > App.java > ...

```java
import java.util.Scanner;

public class App {
    Run | Debug
    public static void main(String[] args) throws Exception {
        System.out.println(x:"Labwork2 from Java");

        Storage.initializeHardwarePorts();

        AxisX axisX = new AxisX();
        int op = -1;
        Scanner scan = new Scanner(System.in);
        while (op != 0){
            System.out.println(x:"Enter an option:");
            op = scan.nextInt();
            switch(op){
                case 1: axisX.moveForward(); break;
                case 2: axisX.moveBackward(); break;
                case 3: axisX.stop(); break;
                default: break;
            }
        }
        scan.close();
    }
}

```



THE CLASS MECHANISM

Create a new class Mechanism.java

Welcome | App.java | Mechanism.java | Axis.java | Storage.java

src > Mechanism.java > Mechanism

```
1
2   public class Mechanism {
3
4       public void ledOn(int ledNumber){
5           Storage.ledOn(ledNumber);
6       }
7
8       public void ledsOff(){
9           Storage.ledsOff();
10      }
11
12      public boolean switch1Pressed(){
13          // todo for now returns false
14          if (Storage.getSwitch1() == 1) return true;
15          return false;
16      }
17
18      public boolean switch2Pressed(){
19          // todo for now returns false
20          return false;
21      }
22
23      public boolean bothSwitchesPressed(){
24          // todo for now returns false
25          return false;
26      }
27
28      public void putPartInCell(){
29          // todo
30      }
31
32      public void takePartFromCell(){
33          //todo
34      }
35  }
36
```

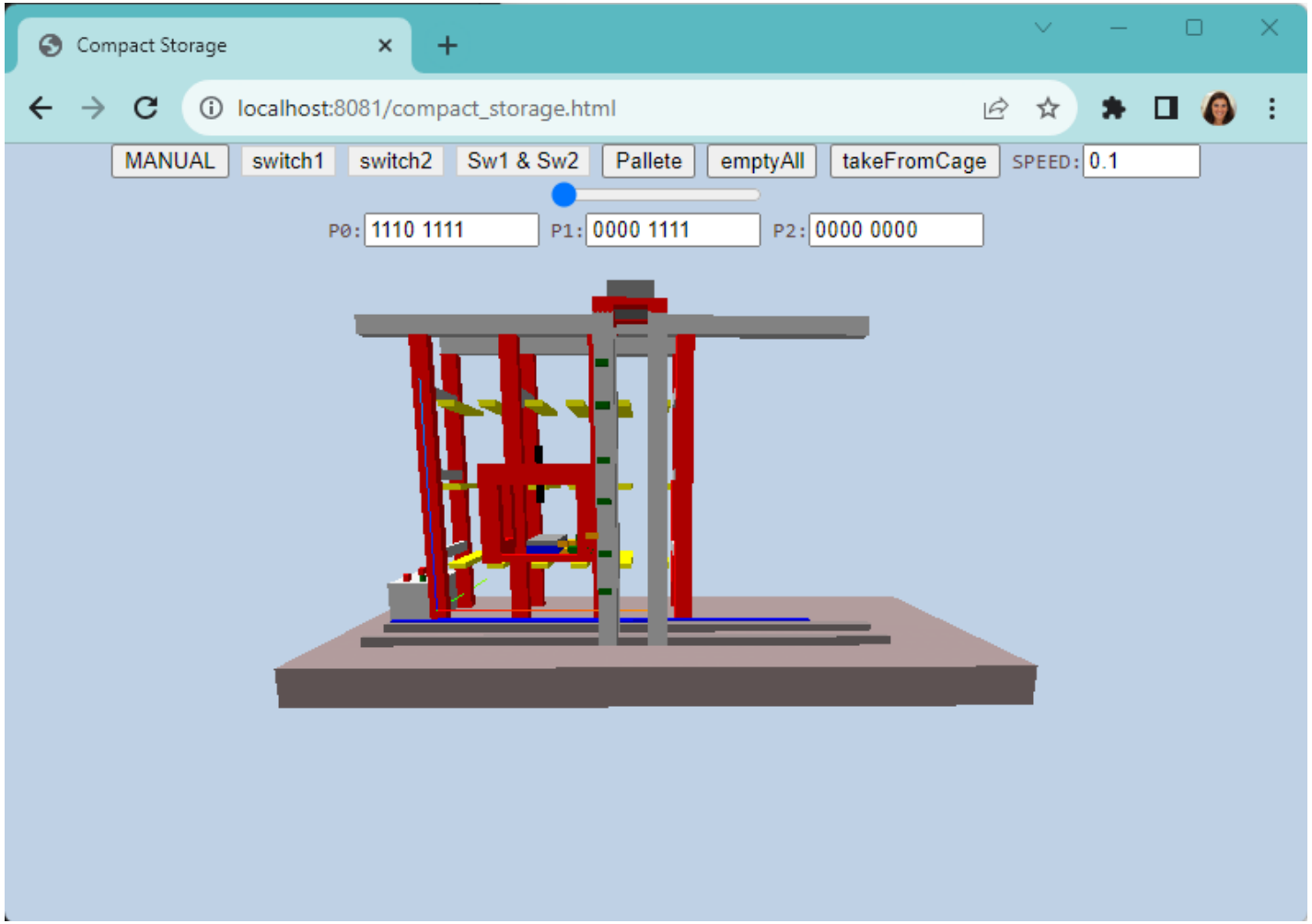
The class mechanism deals with:
- the LEDs
- the switches
- the movements to insert and take parts from cell

Remember, some of these methods might be static, others not

Static methods do not use any instance variables of any object of the class they are defined in. Static methods take all the data from parameters and compute something from those parameters, with no reference to variables.

https://www.tutorialspoint.com/Java-static-method

IDENTIFICATION OF NEEDED JAVA THREADS

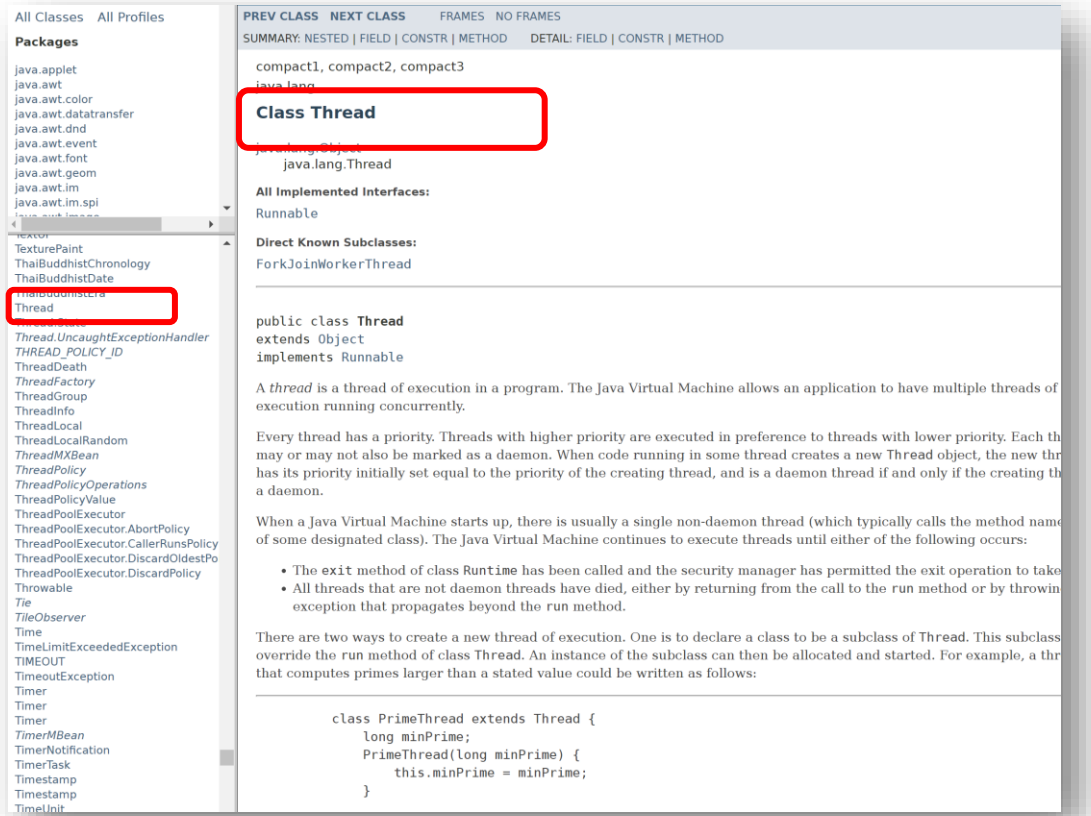


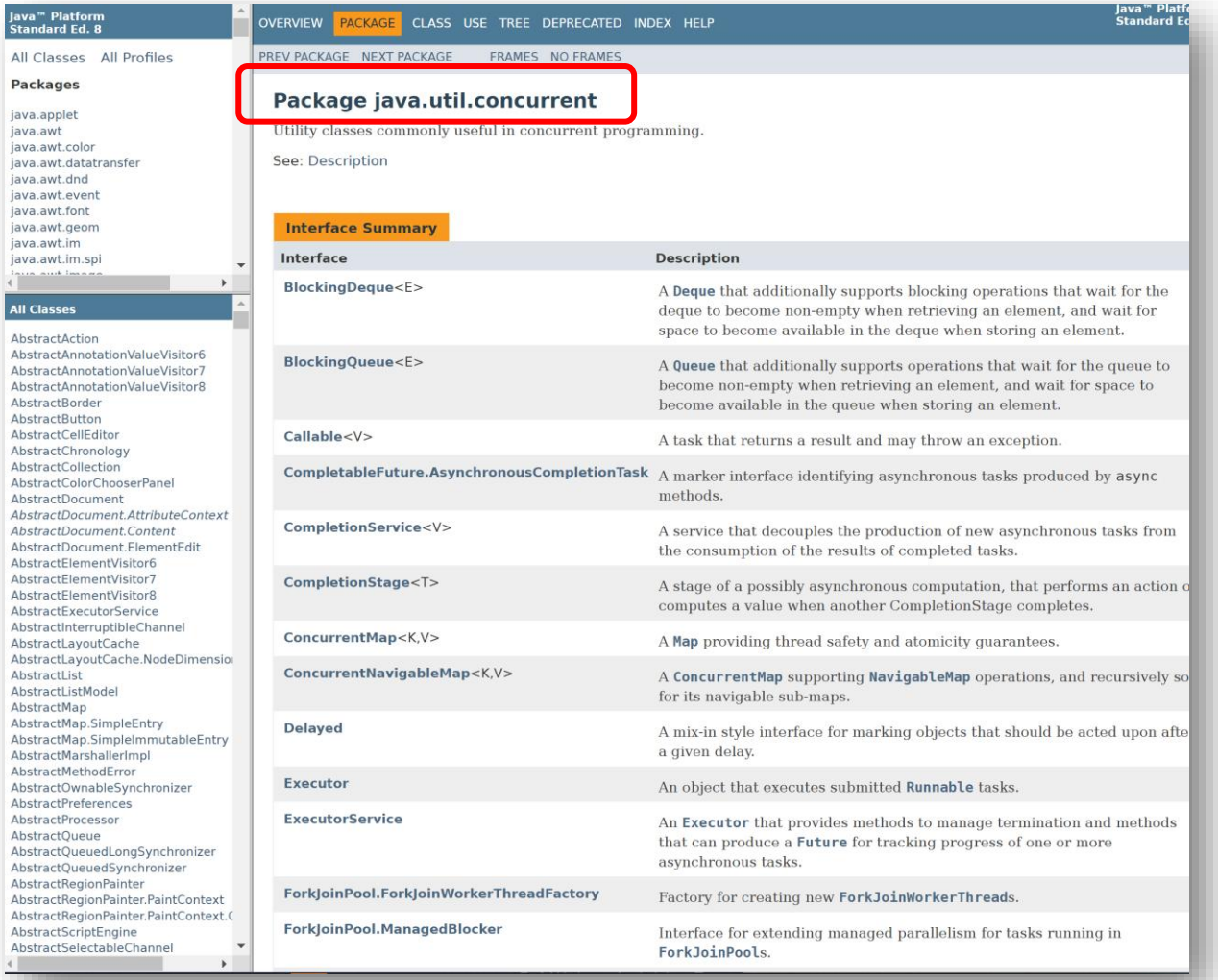
- Start thinking of which THREADS are needed to be executed synchronously or in parallel.
- Tip: start by implementing **thread_calibrate**, **thread_gotoX** and **thread_gotoZ**…
- Draw a first diagram illustrating the threads and the interactions between them.

TO STUDY….

Check….

https://docs.oracle.com/javase/8/docs/api/index.html





https://docs.oracle.com/javase/8/docs/api/index.html?java/util/concurrent/package-summary.html

KEEP UP THE GOOD WORK!